JAVA PROGRAMMING / DSA

Chapter 49: Advanced DP Variants

Personal Notes

Chapter 32 introduced DP through the classical patterns — 1D and 2D tables, memoization, tabulation. That handles most interview problems. But some DPs need STATE that's more clever than a single index or index-pair. Sometimes state is a SUBSET of items (bitmask DP). Sometimes it's the DIGITS built so far in a number (digit DP). Sometimes the recurrence has huge n and needs MATRIX EXPONENTIATION. And sometimes the state lives on a TREE.

This chapter covers five advanced DP variants that unlock the hardest interview problems: BITMASK DP (state as a bitmap of choices), DIGIT DP (counting numbers by digit properties), MATRIX EXPONENTIATION (O(log n) recurrences), INTERVAL DP (split-then-merge on ranges), and TREE DP (subtree aggregations). Each is a small technique on top of what you already know — but each unlocks a family of problems that basic DP can't touch.

1. Quick Recap — Basic DP from Ch 32

The three questions to define any DP:

1. What is the STATE? (What variables uniquely identify a subproblem)
2. What is the RECURRENCE? (How to compute state from smaller states)
3. What is the BASE CASE? (Trivial subproblems with known answers)

The advanced variants all follow this same template — the only thing that changes is WHAT the state looks like.

2. Bitmask DP — State as a Subset

When state is "which items have I used?", represent that subset as a single INTEGER using bits (Ch 38). For n items, there are 2^n possible subsets, each an integer from 0 to 2^n - 1. This gives you exponential state — but for small n (≤ 20), that's still very tractable.

The Setup

```
Items:    A  B  C  D          (positions 0, 1, 2, 3)

Bitmask    Subset represented      As integer
────────   ──────────────────      ──────────
0000       {}                      0
0011       {A, B}                  3
0101       {A, C}                  5
1111       {A, B, C, D}            15
```

```
Total subsets for n items: 2^n
For n = 20: about 1 million states — fits in an int easily.
```

The Universal Bitmask DP Template

```
// dp[mask] or dp[mask][extra] = answer for the subset represented by mask
int[] dp = new int[1 << n];

for (int mask = 0; mask < (1 << n); mask++) {
    for (int i = 0; i < n; i++) {
        if ((mask & (1 << i)) != 0) {          // item i is in the subset
            int prev = mask ^ (1 << i);         // remove item i
            dp[mask] = min(dp[mask], dp[prev] + cost(prev, i));
        }
    }
}
return dp[(1 << n) - 1];   // answer for the FULL set
```

The pattern: For each mask (subset), consider every item i IN the mask. dp[mask] = best answer coming FROM the smaller subset (mask without item i) PLUS the cost of adding i. Iterating masks in increasing order guarantees smaller subsets are processed first.

3. Bitmask DP — Traveling Salesman (TSP)

Given N cities and pairwise distances, find the shortest tour that visits every city exactly once and returns to the start. Brute force is O(n!) — impossible for n > 12. Bitmask DP does it in O($n^2 \times 2^n$) — tractable up to n ≈ 20.

State

- mask — the set of cities visited so far
- i — the current city
- dp[mask][i] = minimum cost to reach city i, having visited exactly the cities in mask

```
public int tsp(int[][] dist) {
    int n = dist.length;
    int[][] dp = new int[1 << n][n];
    for (int[] row : dp) Arrays.fill(row, Integer.MAX_VALUE);

    dp[1][0] = 0;   // start at city 0, visited set = {0}

    for (int mask = 1; mask < (1 << n); mask++) {
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) == 0) continue;
            if (dp[mask][i] == Integer.MAX_VALUE) continue;

            for (int j = 0; j < n; j++) {
```

```
                if ((mask & (1 << j)) != 0) continue;   // j already
visited
                int newMask = mask | (1 << j);
                dp[newMask][j] = Math.min(dp[newMask][j], dp[mask][i] +
dist[i][j]);
            }
        }
    }

    // Answer: minimum cost after visiting all, plus return to start
    int ans = Integer.MAX_VALUE;
    int full = (1 << n) - 1;
    for (int i = 1; i < n; i++) {
        if (dp[full][i] != Integer.MAX_VALUE) {
            ans = Math.min(ans, dp[full][i] + dist[i][0]);
        }
    }
    return ans;
}

// Time: O(n² × 2^n)
// Space: O(n × 2^n)
```

4. Bitmask DP — Assign N Tasks to N Workers

N tasks, N workers, cost[i][j] to assign task j to worker i. Find the minimum total cost of assigning each worker to exactly one task. Bitmask DP: state is (which tasks are assigned so far).

```
public int minCost(int[][] cost) {
    int n = cost.length;
    int[] dp = new int[1 << n];
    Arrays.fill(dp, Integer.MAX_VALUE);
    dp[0] = 0;

    for (int mask = 0; mask < (1 << n); mask++) {
        if (dp[mask] == Integer.MAX_VALUE) continue;
        int assigned = Integer.bitCount(mask);   // worker index
        if (assigned == n) continue;

        for (int j = 0; j < n; j++) {
            if ((mask & (1 << j)) != 0) continue;
            int newMask = mask | (1 << j);
            dp[newMask] = Math.min(dp[newMask], dp[mask] + cost[assigned]
[j]);
        }
    }

    return dp[(1 << n) - 1];
```

```
}
```

The trick: Integer.bitCount(mask) tells you how many tasks are already assigned — that's implicitly the WORKER we're about to schedule. State reduces from O(n × 2^n) to O(2^n).

5. Digit DP — Counting Numbers with Digit Properties

"How many integers from 1 to N have digit sum equal to K?" "How many numbers up to N contain no 4s?" These are DIGIT DP problems — you build numbers digit-by-digit and track the constraints via DP state.

The State

- pos — current digit position (from most significant to least)
- tight — whether we're still bounded by the digits of N (a boolean)
- Extra state — depends on the property (running sum, last digit, etc.)

The Template

```
String digits;
Integer[][][] memo;

public int solve(int pos, int sum, boolean tight) {
    if (pos == digits.length()) return sum == TARGET ? 1 : 0;
    if (!tight && memo[pos][sum][0] != null) return memo[pos][sum][0];

    int limit = tight ? digits.charAt(pos) - '0' : 9;
    int result = 0;
    for (int d = 0; d <= limit; d++) {
        result += solve(pos + 1, sum + d, tight && (d == limit));
    }

    if (!tight) memo[pos][sum][0] = result;
    return result;
}

public int countUpTo(int n) {
    digits = String.valueOf(n);
    memo = new Integer[digits.length()][100][2];
    return solve(0, 0, true);
}
```

Example — Count Numbers ≤ N with Digit Sum = K

```
     N = 25, K = 4

     Build numbers digit by digit:
        d=0 → tens=0: units can be 0..9 → count where sum=4: just "4"
→ 1 number
```

```
        d=1 → tens=1: units can be 0..9 → count where 1+unit=4: unit=3
→ 1 number
        d=2 → tens=2:
              tight remains, units can be 0..5
              count where 2+unit=4: unit=2
→ 1 number

     Total: 3 numbers (4, 13, 22)
```

The tight flag is critical: When tight is TRUE, we can only choose digits up to the corresponding digit of N (to stay ≤ N). Once we pick a digit LESS than the corresponding digit, tight becomes FALSE for all subsequent positions — we can then freely choose 0-9. This bookkeeping is what makes digit DP work for large N.

6. Digit DP — Numbers Without a Certain Digit

LeetCode 902 (Numbers At Most N Given Digit Set) is a canonical example. Count how many numbers ≤ N can be built using ONLY certain digits.

```
public int atMostNGivenDigitSet(String[] digits, int n) {
    String s = String.valueOf(n);
    int len = s.length();
    int d = digits.length;

    // Numbers with strictly FEWER digits than n
    int total = 0;
    for (int i = 1; i < len; i++) {
        total += (int) Math.pow(d, i);
    }

    // Numbers with the SAME digit count, from MSB to LSB
    for (int i = 0; i < len; i++) {
        char cur = s.charAt(i);
        boolean found = false;
        for (String x : digits) {
            char dc = x.charAt(0);
            if (dc < cur) {
                total += (int) Math.pow(d, len - i - 1);
            } else if (dc == cur) {
                found = true;
            }
        }
        if (!found) return total;
    }
    return total + 1;   // n itself is buildable
}

// Time: O(len × d)
```

7. Matrix Exponentiation — Recurrences in O(log n)

Standard Fibonacci runs in O(n). But if the recurrence has huge n (say n = 10^18), even O(n) is too slow. MATRIX EXPONENTIATION reduces linear recurrences to matrix multiplication — and matrix power can be computed in O(log n) with fast exponentiation (Ch 47).

The Fibonacci Example

```
     Recurrence:  F(n) = F(n-1) + F(n-2)

     Matrix form:
        [F(n+1)]   [1 1]   [F(n)]
        [ F(n) ] = [1 0] × [F(n-1)]

     Applying k times:
        [F(k+1)]   [1 1]^k   [F(1)]
        [ F(k) ] = [1 0]   × [F(0)]

     Compute the matrix to the kth power in O(log k) — same as fast power
on integers.
```

Full Code

```
public long fibonacci(long n) {
    if (n == 0) return 0;
    long[][] base = {{1, 1}, {1, 0}};
    long[][] result = matPow(base, n);
    return result[1][0];    // this is F(n)
}

private long[][] matPow(long[][] mat, long p) {
    long[][] result = {{1, 0}, {0, 1}};   // identity matrix
    while (p > 0) {
        if ((p & 1) == 1) result = multiply(result, mat);
        mat = multiply(mat, mat);
        p >>= 1;
    }
    return result;
}

private long[][] multiply(long[][] a, long[][] b) {
    long[][] r = new long[2][2];
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            for (int k = 0; k < 2; k++)
                r[i][j] += a[i][k] * b[k][j];   // add MOD if computing
modulo
    return r;
}
```

```
// Time: O(log n)  |  Space: O(1)  (fixed-size matrices)
```

Any LINEAR recurrence can be matrix-exponentiated: If the next state depends linearly on some fixed number of previous states, you can express one step as multiplication by a fixed matrix. Then k steps become that matrix to the k-th power. Works for Fibonacci, tribonacci, staircase-with-moves-of-{1,2,3}, and many DP recurrences with huge n.

8. Interval DP — Split-Then-Merge

Some problems ask about the best way to "merge" or "split" a range. State is dp[i][j] = best answer for the range [i, j]. The recurrence tries every possible split point k in (i, j) and combines subrange answers.

The Template

```
for (int len = 2; len <= n; len++) {                    // interval length
    for (int i = 0; i + len - 1 < n; i++) {              // left endpoint
        int j = i + len - 1;                              // right
endpoint
        dp[i][j] = INF;
        for (int k = i; k < j; k++) {                     // split point
            dp[i][j] = Math.min(dp[i][j],
                                dp[i][k] + dp[k + 1][j] + costOfMerge(i,
k, j));
        }
    }
}
return dp[0][n - 1];
```

Classic — Matrix Chain Multiplication

```
// dims[i-1] × dims[i] is the size of matrix i (1-indexed)
public int matrixChainOrder(int[] dims) {
    int n = dims.length - 1;
    int[][] dp = new int[n][n];

    for (int len = 2; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            dp[i][j] = Integer.MAX_VALUE;
            for (int k = i; k < j; k++) {
                int cost = dp[i][k] + dp[k + 1][j] + dims[i] * dims[k + 1]
* dims[j + 1];
                dp[i][j] = Math.min(dp[i][j], cost);
            }
        }
    }
    return dp[0][n - 1];
}
```

```
// Time: O(n³)  |  Space: O(n²)
```

Classic — Burst Balloons (LeetCode 312)

The famous variant: given balloons with values, popping balloon i earns nums[i-1] × nums[i] × nums[i+1]. Find the MAX coins by popping in the optimal order. Trick: think REVERSE — pop the LAST balloon last. dp[i][j] = max coins from bursting all balloons strictly between i and j.

```
public int maxCoins(int[] nums) {
    int n = nums.length;
    int[] arr = new int[n + 2];
    arr[0] = arr[n + 1] = 1;
    for (int i = 0; i < n; i++) arr[i + 1] = nums[i];

    int[][] dp = new int[n + 2][n + 2];

    for (int len = 2; len <= n + 1; len++) {
        for (int i = 0; i + len <= n + 1; i++) {
            int j = i + len;
            for (int k = i + 1; k < j; k++) {
                dp[i][j] = Math.max(dp[i][j],
                                    dp[i][k] + dp[k][j] + arr[i] * arr[k]
* arr[j]);
            }
        }
    }
    return dp[0][n + 1];
}

// Time: O(n³)
```

9. Tree DP — State on a Tree

When the problem is on a tree, DP state is often "the best answer for the SUBTREE rooted at node u." Compute via DFS — process children first, then combine at the parent.

Classic — House Robber III

LeetCode 337. Given a binary tree of house values, rob houses without robbing any TWO connected houses. Return max robbed.

```
public int rob(TreeNode root) {
    int[] result = dfs(root);
    return Math.max(result[0], result[1]);
}

// Returns {without_root, with_root}
private int[] dfs(TreeNode node) {
```

```
    if (node == null) return new int[]{0, 0};
    int[] left = dfs(node.left);
    int[] right = dfs(node.right);

    // Skip current: children can be robbed or not (take max of each)
    int without = Math.max(left[0], left[1]) + Math.max(right[0],
right[1]);

    // Rob current: children CANNOT be robbed
    int with = node.val + left[0] + right[0];

    return new int[]{without, with};
}

// Time: O(n)  |  Space: O(h) for recursion
```

The pattern: For each node, compute a TUPLE representing multiple possibilities (rob this / skip this, take path / don't take path, etc.). Combine children's tuples to compute the parent's. Root's tuple is the answer. This generalizes to "pick k nodes such that no two are adjacent," "tree diameter," "count paths through each node," and many more.

Another Classic — Diameter of a Binary Tree

```
class Solution {
    int diameter = 0;

    public int diameterOfBinaryTree(TreeNode root) {
        depth(root);
        return diameter;
    }

    private int depth(TreeNode node) {
        if (node == null) return 0;
        int left  = depth(node.left);
        int right = depth(node.right);
        diameter = Math.max(diameter, left + right);   // path through
this node
        return 1 + Math.max(left, right);              // depth to pass up
    }
}
```

10. Knapsack Variants — A Quick Tour

Ch 32 covered 0/1 knapsack (each item chosen 0 or 1 times). Three variants show up in interviews — each is a small tweak to the recurrence.

10.1 0/1 Knapsack (Recap)

```
for (int i = 1; i <= n; i++)
    for (int w = 0; w <= W; w++) {
        dp[i][w] = dp[i-1][w];
        if (weight[i] <= w)
            dp[i][w] = Math.max(dp[i][w], dp[i-1][w - weight[i]] +
value[i]);
    }
// Each item used at most once.
```

10.2 Unbounded Knapsack — Each Item Any Number of Times

```
for (int w = 0; w <= W; w++)
    for (int i = 0; i < n; i++)
        if (weight[i] <= w)
            dp[w] = Math.max(dp[w], dp[w - weight[i]] + value[i]);
// Coin change (min/max coins), rod cutting, complete knapsack.
```

10.3 Bounded Knapsack — At Most k[i] Copies of Item i

Naïvely: run 0/1 knapsack treating each copy as a separate item. Better: BINARY GROUPING — represent k[i] copies as log(k[i]) groups of sizes 1, 2, 4, …, leftover — turning it into 0/1 knapsack with O(log k) items per original.

10.4 Subset Sum — Boolean Variant

```
// Can we form sum W using some subset?
boolean[] dp = new boolean[W + 1];
dp[0] = true;
for (int w : weights) {
    for (int s = W; s >= w; s--) {
        dp[s] = dp[s] || dp[s - w];
    }
}
return dp[W];

// Classic partition problems: LeetCode 416, 494, 698, etc.
```

11. When to Reach for Each Variant

If the problem asks…	Use
State = which items are chosen (n ≤ 20)	**Bitmask DP**
Count numbers with digit properties	**Digit DP**

If the problem asks...	Use
Linear recurrence with huge n (10^18)	**Matrix exponentiation**
Best split/merge on a range	**Interval DP**
Best answer for a subtree	**Tree DP (DFS with tuple return)**
Assignment / matching (small n)	**Bitmask DP**
Traveling salesman (n ≤ 20)	**Bitmask DP**
Fibonacci-like recurrence mod prime	**Matrix exponentiation**
Matrix chain / burst balloons / palindrome partition	**Interval DP**
Robbing/rearranging on a tree	**Tree DP with (choose/skip) tuple**
Choose items with quantity limits	**Knapsack variants (bounded, unbounded)**

12. Complexity Comparison

Variant	Time	Space
Bitmask DP	O(n × 2^n) or O(n^2 × 2^n)	O(2^n) or O(n × 2^n)
Digit DP	O(len × states × 10)	O(len × states × 2)
Matrix exponentiation	O(k^3 × log n)	O(k^2) (k = matrix size)
Interval DP	O(n^3)	O(n^2)
Tree DP	O(n)	O(h) recursion + O(n) state
0/1 Knapsack	O(n × W)	O(W) space-optimized
Unbounded Knapsack	O(n × W)	O(W)

> **Bitmask DP scale limits:** n = 20 gives 2^20 ≈ 1 million states — fine. n = 25 is 33 million — borderline. n = 30 is 1 billion — too slow. If n > 20 and the state is "subset of items," bitmask DP won't work; you need a different structure (like greedy or approximation).

13. Classic Problem 1 — Shortest Path Visiting All Nodes

LeetCode 847. Given an undirected connected graph, find the shortest path visiting EVERY node (nodes can be revisited). BFS + bitmask state — classic bitmask problem.

```
public int shortestPathLength(int[][] graph) {
    int n = graph.length;
    int all = (1 << n) - 1;
    Queue<int[]> q = new ArrayDeque<>();
    boolean[][] visited = new boolean[n][1 << n];

    for (int i = 0; i < n; i++) {
        q.offer(new int[]{i, 1 << i, 0});
        visited[i][1 << i] = true;
    }

    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int node = cur[0], mask = cur[1], dist = cur[2];
        if (mask == all) return dist;

        for (int next : graph[node]) {
            int newMask = mask | (1 << next);
            if (!visited[next][newMask]) {
                visited[next][newMask] = true;
                q.offer(new int[]{next, newMask, dist + 1});
            }
        }
    }
    return -1;
}

// Time: O(n × 2^n)  |  Space: O(n × 2^n)
```

14. Classic Problem 2 — Partition Equal Subset Sum

LeetCode 416. Given a set of positive integers, can you partition it into two subsets with equal sums? Reduces to: can we form sum = total/2 using some subset? Boolean subset-sum DP.

```
public boolean canPartition(int[] nums) {
    int total = 0;
    for (int n : nums) total += n;
    if (total % 2 != 0) return false;
    int target = total / 2;

    boolean[] dp = new boolean[target + 1];
    dp[0] = true;
    for (int n : nums) {
        for (int s = target; s >= n; s--) {
            dp[s] = dp[s] || dp[s - n];
        }
    }
    return dp[target];
```

```
}
```

15. Classic Problem 3 — Binary Tree Maximum Path Sum

LeetCode 124. Find the max sum of any path in a binary tree (can start and end at any nodes, doesn't need root). Classic tree DP — a node's DFS returns "best straight-line path DOWNWARD" while a global variable tracks best-path-through-this-node.

```
class Solution {
    int best = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
        dfs(root);
        return best;
    }

    private int dfs(TreeNode node) {
        if (node == null) return 0;
        int left  = Math.max(0, dfs(node.left));   // discard negatives
        int right = Math.max(0, dfs(node.right));

        best = Math.max(best, left + right + node.val);   // path through
this node
        return node.val + Math.max(left, right);           // straight-
down path
    }
}
```

16. Classic Problem 4 — Fibonacci for Huge N

LeetCode 509 with n up to 10^18. Standard O(n) DP is too slow — matrix exponentiation is the answer.

The complete implementation is in Section 7. Time: O(log n). Space: O(1).

17. Common Mistakes

Mistake 1: Not initializing bitmask DP arrays

```
int[] dp = new int[1 << n];        // Java defaults to 0 — is 0 valid or
does it mean "impossible"?
Arrays.fill(dp, Integer.MAX_VALUE); // ✓ mark impossible states explicitly
dp[0] = 0;                           // set base case
```

Mistake 2: Forgetting the tight flag in digit DP

Without the tight tracking, you count numbers up to 10^len instead of up to N. The memoization key must NOT include tight (it changes per call), but the recursion must always pass tight forward.

Mistake 3: Not handling MOD in matrix multiplication

For big Fibonacci mod problems, EVERY intermediate multiplication and addition in the matrix multiply must be modded. Miss even one and results silently overflow.

Mistake 4: Wrong loop order in interval DP

```
// WRONG — computes dp[i][j] before dp[i][k] and dp[k+1][j] are ready
for (int i = 0; i < n; i++)
    for (int j = i; j < n; j++) ...

// RIGHT — iterate by INCREASING LENGTH, so subranges are done first
for (int len = 2; len <= n; len++)
    for (int i = 0; i + len - 1 < n; i++) { int j = i + len - 1; ... }
```

Mistake 5: Overflowing bitmask calculations

```
int mask = 1 << 30;    // OK
int mask = 1 << 32;    // ✗ overflows int silently — use 1L << 32 for
large sizes
```

Mistake 6: Tree DP without post-order thinking

Tree DP MUST compute children before parents. Use post-order DFS. Doing it in the wrong order gives wrong results — a common bug when converting to iterative BFS.

Mistake 7: Using bitmask DP when n > 20

2^20 = 1M states — OK. 2^25 = 33M — TLE. 2^30 = 1B — impossible. If n > 20 and you're reaching for bitmask DP, step back — likely wrong approach.

18. Best Practices

- For bitmask DP, verify n ≤ 20 before committing to the approach
- Initialize DP arrays to "impossible" (Integer.MAX_VALUE / MIN_VALUE), then set base case explicitly
- Digit DP: use 3D memo [pos][sum-or-state][tight] — memoize only when NOT tight
- Matrix exp: always add MOD to every multiplication and addition for big-answer problems
- Interval DP: iterate by INCREASING length (len = 2, 3, …, n)
- Tree DP: return a TUPLE (e.g., {without, with}) from DFS to combine multiple states cleanly
- For big n, always ask "is my recurrence linear? — matrix exp might apply"
- Test on tiny inputs (n = 2 or 3) to verify correctness before scaling

19. Quick Reference

Concept	Meaning
Bitmask DP	State = subset of items as an integer; n ≤ 20
Digit DP	Build numbers digit by digit; tight flag critical
Matrix exponentiation	Linear recurrence in O(log n) via matrix power
Interval DP	dp[i][j] = best for range [i, j]; iterate by length
Tree DP	DFS returning a tuple for each subtree
0/1 knapsack	Each item taken 0 or 1 times
Unbounded knapsack	Each item taken any number of times
Bounded knapsack	Each item at most k[i] times; binary grouping trick
Subset sum	Boolean DP — can we form target with a subset?
tight flag	Digit DP — whether we're still bounded by N
Post-order DFS	Required for tree DP — children before parent

20. Practice Problems

Practice in order — bitmask first (most common), then digit DP, then interval and tree.

Bitmask DP

4. Shortest Path Visiting All Nodes (LeetCode 847).
5. Partition to K Equal Sum Subsets (LeetCode 698).
6. Can I Win (LeetCode 464).
7. Traveling Salesman — build TSP from scratch.
8. Number of Ways to Wear Different Hats to Each Other (LeetCode 1434).
9. Assignment Problem — min cost of assigning N tasks to N workers.

Digit DP

10. Count Numbers with Unique Digits (LeetCode 357).
11. Numbers At Most N Given Digit Set (LeetCode 902).
12. Non-negative Integers without Consecutive Ones (LeetCode 600).
13. Count of Integers (LeetCode 2719).

Matrix Exponentiation

14. Fibonacci Number with n up to 10^18.
15. Climbing Stairs with n up to 10^18.
16. Count number of ways to tile a 2 × N board.

Interval DP

17. Matrix Chain Multiplication.
18. Burst Balloons (LeetCode 312).
19. Minimum Cost to Merge Stones (LeetCode 1000).
20. Predict the Winner (LeetCode 486).
21. Palindrome Partitioning II (LeetCode 132).

Tree DP

22. House Robber III (LeetCode 337).
23. Binary Tree Maximum Path Sum (LeetCode 124).
24. Diameter of Binary Tree (LeetCode 543).
25. Longest Univalue Path (LeetCode 687).
26. Sum of Distances in Tree (LeetCode 834) — reroute technique.

Knapsack Variants

27. Partition Equal Subset Sum (LeetCode 416).
28. Target Sum (LeetCode 494).
29. Coin Change II (LeetCode 518) — unbounded.
30. Ones and Zeroes (LeetCode 474) — 2D 0/1 knapsack.

21. Final Takeaway

Basic DP handles most interview problems. Advanced DP handles the last 20% — the trickiest ones that separate strong candidates from great ones. The five variants in this chapter cover essentially every hard DP problem you'll encounter at Google: bitmask DP for subset problems (n ≤ 20), digit DP for counting-by-digits, matrix exponentiation for huge n, interval DP for split/merge, and tree DP for subtree aggregations.

For interviews, master three variants: BITMASK DP (Shortest Path Visiting All Nodes, TSP), INTERVAL DP (Burst Balloons, Matrix Chain), and TREE DP (Max Path Sum, House Robber III). Digit DP and matrix exponentiation are lower-frequency but high-signal — knowing them puts you ahead of most candidates.

> **Key Takeaway:** Advanced DP = same recurrence framework, richer state. BITMASK DP: state is a subset (n ≤ 20). DIGIT DP: build numbers digit-by-digit with a tight flag. MATRIX EXPONENTIATION: linear recurrences in O(log n) via matrix power. INTERVAL DP: dp[i][j] over ranges, iterate by length. TREE DP: DFS returning tuples for subtree states. Knapsack variants: 0/1, unbounded, bounded (binary grouping), subset-sum. Choose the variant by asking: what does state actually represent?

22. What's Next?

With advanced DP covered, the DSA + algorithmic patterns toolkit is thoroughly complete. Remaining directions:

- System Design fundamentals — the natural big step for senior/staff interviews
- Concurrency & Multithreading in depth — locks, atomics, executor pools
- Max Flow / Min Cut — Ford-Fulkerson, Edmonds-Karp
- Computational Geometry — convex hull, line sweep, closest pair
- Suffix Arrays / Automata — advanced string data structures
- Behavioral / Leadership interviews — STAR method, Googleyness
- Mock interview practice — put it all together under time pressure